Ain Shams University
Faculty of Engineering
Computer & Systems Department

CSE335s: Operating Systems

CPU Scheduler Simulator Project

Click here to install: https://github.com/amrrnagy/CPU-Scheduler-Simulator/releases/download/v2.0.0/setup.exe

Name	ID
Amr Ahmed Nagy	2300162
Mohamed Ashraf Mohamed	2300475
Ali Ahmed Ayman	2300346
Omar Ahmed Morshed	2300131
Eslam Waleed Ebrahim	2300022
Amr Emad Lotfy	2300379

Presented to: Dr. Sahar M. Mahmoud Haggag

1. Overview:

This project presents a CPU task scheduler simulator designed to emulate the behavior of classical scheduling algorithms: First-Come, First-Served (FCFS), Shortest Job First (SJF), Priority Scheduling, and Round Robin. The application features a robust backend responsible for algorithmic logic, seamlessly integrated with an interactive desktop Graphical User Interface (GUI).

The simulator is built to handle both static execution and a "Live" simulation mode. In live mode, execution time is scaled such that 1 unit of simulation time maps strictly to 1 real-world second. The system dynamically updates a live Gantt chart, computes performance statistics (Average Waiting Time, Average Turnaround Time), and allows for the dynamic insertion of new processes at runtime without interrupting the current execution flow.

2. System Architecture:

To achieve a responsive GUI alongside a time-accurate live simulation, the system utilizes a multithreaded, object-oriented architecture. The design enforces a strict separation of concerns among three primary entities:

1. **The Simulator (Main/GUI Thread):** This is the core controller encompassing the application's state. It manages all user interactions, input validation, and real-time visual updates to the process tables and Gantt chart.
2. **The Scheduler:** A modular component responsible for sorting and organizing the Ready Queue based on the rules of the user-selected algorithm (e.g., sorting by shortest remaining time or priority number).
3. **The Runner (Worker Thread):** This component acts as the emulated CPU. It runs on a separate background thread to ensure that the 1-second delay loops do not block or freeze the Main GUI thread.

These components communicate via shared data structures and synchronization primitives to ensure thread safety when a user dynamically adds a new process during a live run.

3. How the Scheduler Works

The backend logic was designed around the following observations regarding process states:

- The CPU (Runner) only cares about the process currently at the head of the Ready Queue.
- For preemptive algorithms, any newly arriving process must be immediately compared against the current executing process to determine if a context switch is required.

- **Execution Flow:**

  1. When a process is added via the GUI, it is instantiated as a Process object (encapsulating its ID, Arrival Time, Burst Time, and Priority) and added to a central process list.
  2. The Scheduler evaluates the algorithm type. If the algorithm is non-preemptive, the process is placed in the queue and waits for its turn. If preemptive, the Scheduler evaluates if the new process has a stricter condition (e.g., higher priority or shorter remaining time) than the active process.
  3. If preemption occurs, the Runner pushes the current process back into the Ready Queue and immediately begins executing the new head process.
  4. The Runner maintains an internal execution log that records the start and end of every time slice. This log is continuously polled by the GUI to render the live Gantt chart accurately.

4. Key Design Decisions:

Handling the 1-Second Live Mapping Constraint: A major technical challenge was mapping 1 unit of simulation time to exactly 1 second of real time without causing UI unresponsiveness. Relying solely on basic thread-sleep functions on the main application thread would freeze the user interface, preventing the dynamic insertion of new processes.

- **Solution:** We decoupled the timing logic from the UI. The simulation loop runs on a dedicated background thread. It uses asynchronous timers to wait for 1 second per burst unit. After each second, the background thread safely broadcasts a state-update event to the Main GUI thread. This allows the process table and Gantt chart to redraw smoothly while keeping the input fields active for new arrivals.

Dynamic GUI Inputs: To ensure a streamlined user experience, the GUI is designed to conditionally render input fields. For instance, when FCFS is selected, the priority input fields are disabled or hidden, ensuring the user is never prompted for unused information.

Data Structures: A dynamic queue structure was chosen for the Ready Queue. Because the application must frequently insert new processes into the Ready Queue (especially in SJF and Priority scheduling), a heap-based **std::priority_queue** was implemented. This allows the Scheduler to efficiently maintain the correct execution order upon every new arrival with logarithmic time complexity, avoiding the heavy overhead of re-sorting an entire array.

5. GUI Implementation & Screenshots:

The graphical user interface was developed using the **Qt Framework**. Utilizing Qt allowed for a seamless and highly responsive integration between the backend scheduling logic and the interactive frontend.

Specifically, the framework's **Signals and Slots** event-handling mechanism was essential for this project's live simulation requirement. It enabled thread-safe communication between the background worker thread (the Runner) and the main application thread. This ensured that continuous live updates—such as the dynamic drawing of the Gantt chart and the real-time decrements in the Remaining Burst Timetable- were rendered smoothly without ever freezing the interface or blocking the user from dynamically adding new processes.

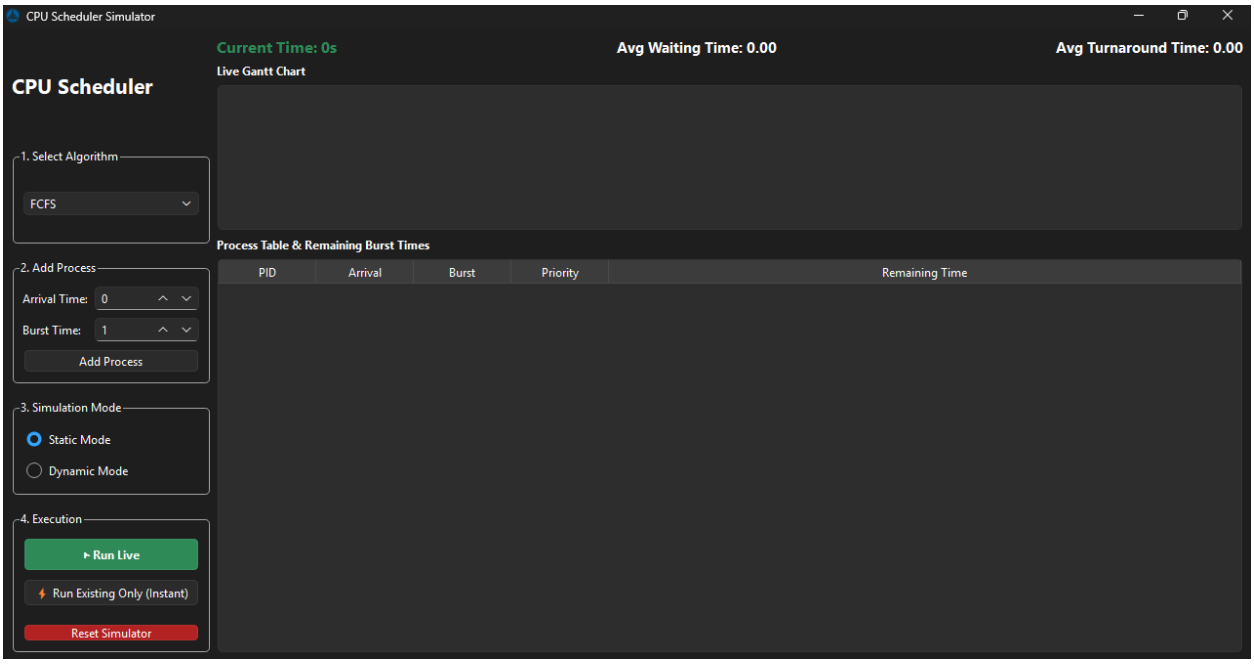

Fig1.Simulator UI

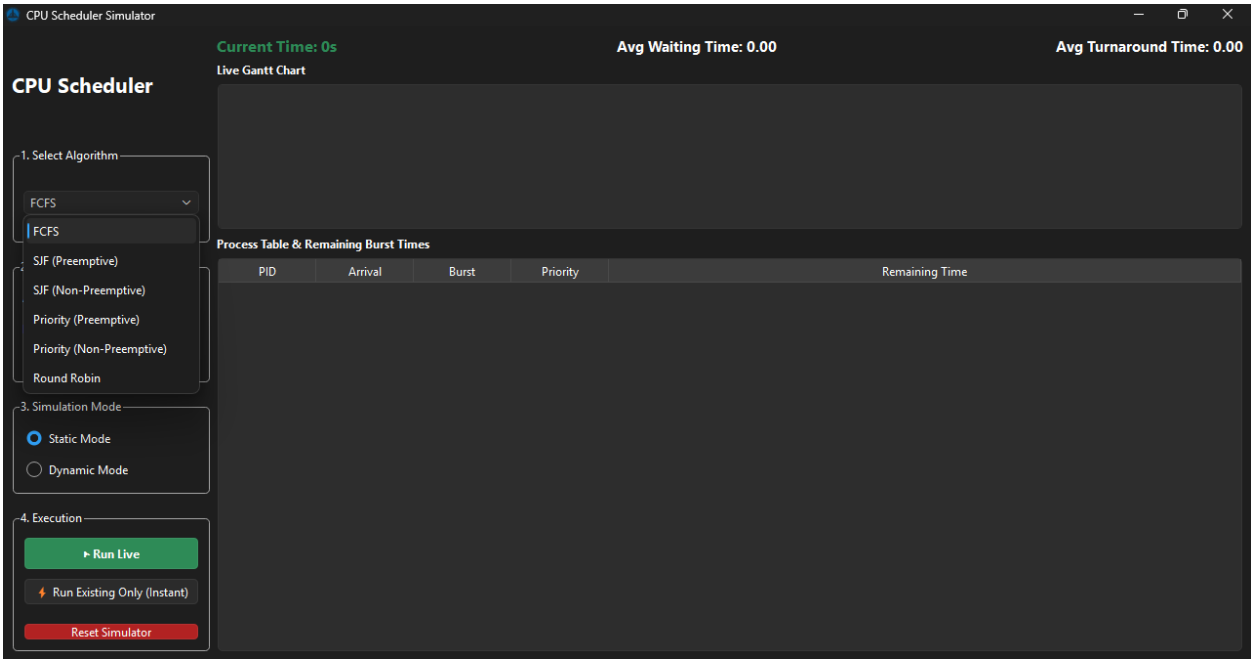

Fig2.User Input

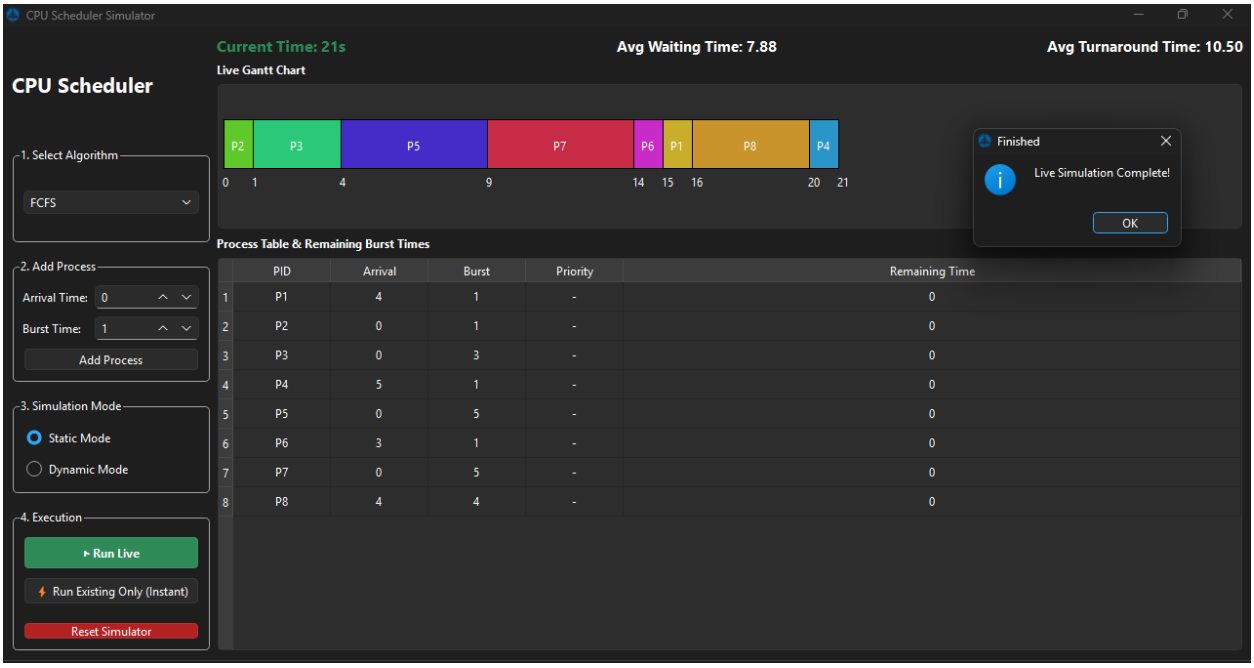

Fig3.Static FCFS Simulation